

Python 小程序 3

以此代码演示了文件操作、JSON 数据处理、面向对象编程和数据持久化的实现

```
import json
import os
from datetime import datetime

class FinanceRecord:
    """财务记录类"""
    def __init__(self, record_type: str, amount: float, description: str, date: str = None):
        self.record_type = record_type # 'income' 或 'expense'
        self.amount = amount
        self.description = description
        self.date = date if date else datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    def to_dict(self) -> dict:
        """转换为字典，用于 JSON 保存"""
        return {
            'type': self.record_type,
            'amount': self.amount,
            'description': self.description,
            'date': self.date
        }
    @staticmethod
    def from_dict(data: dict) -> 'FinanceRecord':
        """从字典创建财务记录对象"""
        return FinanceRecord(
            data['type'],
            data['amount'],
            data['description'],
            data['date']
        )
    def __str__(self) -> str:
        type_str = "收入" if self.record_type == 'income' else "支出"
        return f"{self.date} | {type_str} | ￥{self.amount:.2f} | {self.description}"

class FinanceManager:
    """个人财务管理系统类"""
    def __init__(self, data_file: str = "finance_data.json"):
        self.data_file = data_file
        self.records = []
```

```

        self.load_data()
def load_data(self) -> None:
    """从文件加载数据"""
    if os.path.exists(self.data_file):
        try:
            with open(self.data_file, 'r', encoding='utf-8') as f:
                data = json.load(f)
                self.records = [FinanceRecord.from_dict(record) for record in
data]

            print(f"成功加载 {len(self.records)} 条财务记录")
        except Exception as e:
            print(f"加载数据失败: {e}")
            self.records = []
    else:
        print("未找到数据文件，将创建新文件")
def save_data(self) -> None:
    """保存数据到文件"""
    try:
        with open(self.data_file, 'w', encoding='utf-8') as f:
            json.dump([record.to_dict() for record in self.records], f,
ensure_ascii=False, indent=4)
            print("数据保存成功")
    except Exception as e:
        print(f"保存数据失败: {e}")
def add_record(self) -> None:
    """添加财务记录"""
    try:
        record_type = input("请输入记录类型 (income/支出): ").strip().lower()
        if record_type not in ['income', '支出']:
            print("类型错误! 请输入 'income' 或 '支出'")
            return
        # 统一为英文类型
        record_type = 'income' if record_type == 'income' else 'expense'
        amount = float(input("请输入金额: "))
        if amount <= 0:
            print("金额必须大于 0! ")
            return
        description = input("请输入描述: ").strip()
        if not description:
            description = "无描述"

```

```

        record = FinanceRecord(record_type, amount, description)
        self.records.append(record)
        self.save_data()
        print("记录添加成功! ")
    except ValueError:
        print("金额必须是数字! ")
def show_all_records(self) -> None:
    """显示所有记录"""
    if not self.records:
        print("暂无财务记录! ")
        return
    print("\n" + "="*80)
    print("所有财务记录:")
    print("-"*80)
    for i, record in enumerate(self.records, 1):
        print(f"{i}. {record}")
    print("="*80)
def show_statistics(self) -> None:
    """显示统计信息"""
    if not self.records:
        print("暂无财务记录, 无法统计! ")
        return
    total_income = sum(r.amount for r in self.records if r.record_type ==
'income')
    total_expense = sum(r.amount for r in self.records if r.record_type ==
'expense')
    balance = total_income - total_expense
    print("\n" + "="*50)
    print("财务统计:")
    print(f"总收入: ¥{total_income:.2f}")
    print(f"总支出: ¥{total_expense:.2f}")
    print(f"结余: ¥{balance:.2f}")
    print(f"记录总数: {len(self.records)}")
    print("="*50)
def search_records(self) -> None:
    """搜索记录"""
    keyword = input("请输入搜索关键词: ").strip().lower()
    if not keyword:
        print("关键词不能为空! ")
        return

```

```

found = []
for record in self.records:
    if (keyword in record.description.lower() or
        keyword in record.date or
        keyword in str(record.amount)):
        found.append(record)
if not found:
    print("未找到匹配的记录！")
    return
print(f"\n 找到 {len(found)} 条匹配记录:")
print("-"*80)
for i, record in enumerate(found, 1):
    print(f"{i}. {record}")
print("-"*80)
def run(self) -> None:
    """运行财务管理系统"""
    while True:
        print("\n===== 个人财务管理系统 =====")
        print("1. 添加记录")
        print("2. 查看所有记录")
        print("3. 查看统计信息")
        print("4. 搜索记录")
        print("0. 退出系统")
        try:
            choice = int(input("请输入您的选择: "))
            if choice == 0:
                print("感谢使用，再见！")
                break
            elif choice == 1:
                self.add_record()
            elif choice == 2:
                self.show_all_records()
            elif choice == 3:
                self.show_statistics()
            elif choice == 4:
                self.search_records()
            else:
                print("无效的选择，请重新输入！")
        except ValueError:

```

```
        print("请输入数字! ")

if __name__ == "__main__":
    manager = FinanceManager()
    manager.run()
```